Name: Arya Bailur
Class: T1
Roll No: 08
Subject: Agile Software Development and DevOps Lab

Experiment 3

Aim: To containerise the application using docker

This experiment maps to LO4.
LO4:Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.

Theory

Docker is an open-source platform used to create, deploy, and run applications inside lightweight containers. A container packages an application along with all its required dependencies, libraries, and configuration files so that it can run consistently on any system.

Docker uses **containerization technology**, which allows multiple applications to run separately on the same machine without interfering with each other. Containers are faster and more lightweight compared to traditional virtual machines because they share the host operating system kernel.

Docker Components:

1. **Docker Image**
   A Docker image is a read-only template used to create containers. It contains the application code, libraries, and dependencies required to run an application.
2. **Docker Container**
   A container is a running instance of a Docker image. It provides an isolated environment where applications can execute.
3. **Docker Hub**
   Docker Hub is a cloud-based repository where Docker images can be stored, shared, and downloaded.
4. **Docker Compose**
   Docker Compose is a tool used to define and manage multiple Docker containers as services using a YAML configuration file.

Yml Code

```
version: "3.9"


services:
  web_server:
    image: nginx:latest
    container_name: web_server
    ports:
      - "8090:80"
    restart: unless-stopped


  mysql_db:
    image: mysql:8.0
    container_name: mysql_db
    ports:
      - "3307:3306"
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
      MYSQL_DATABASE: mydb
      MYSQL_USER: appuser
      MYSQL_PASSWORD: apppass
    volumes:
      - mysql_data:/var/lib/mysql
```

```
    restart: unless-stopped

  redis_cache:
    image: redis:latest
    container_name: redis_cache
    ports:
      - "6379:6379"
    restart: unless-stopped

  adminer_ui:
    image: adminer:latest
    container_name: adminer_ui
    ports:
      - "8085:8080"
    depends_on:
      - mysql_db
    restart: unless-stopped

volumes:
  mysql_data:
```

Services Created Using Docker Compose

In this experiment, a `docker-compose.yml` file was created to define and run multiple services together.

The services created were:

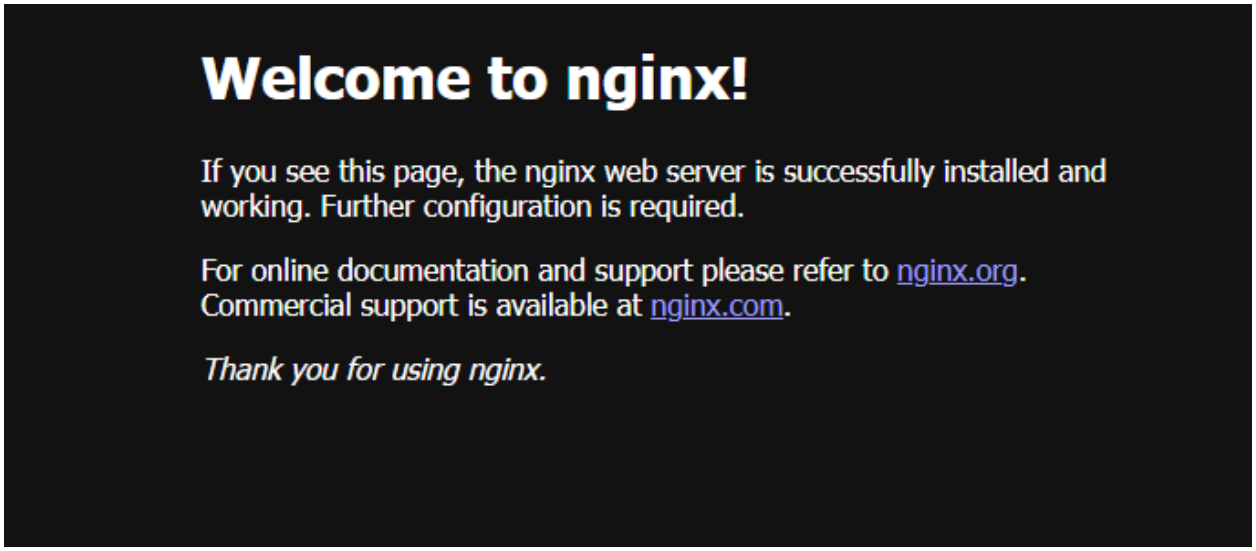
1. Web Service (Nginx)

- **Image used:** `nginx:latest`
- **Container name:** `web_server`
- **Port:** `8090:80`

Nginx is a web server used to serve web pages and handle client requests. In this experiment, it was used as the web service running inside a Docker container.

Access URL:

http://localhost:8090



2. Database Service (MySQL)

- **Image used:** `mysql:8.0`
- **Container name:** `mysql_db`
- **Port:** `3307:3306`

MySQL is a relational database management system used to store and manage application data.

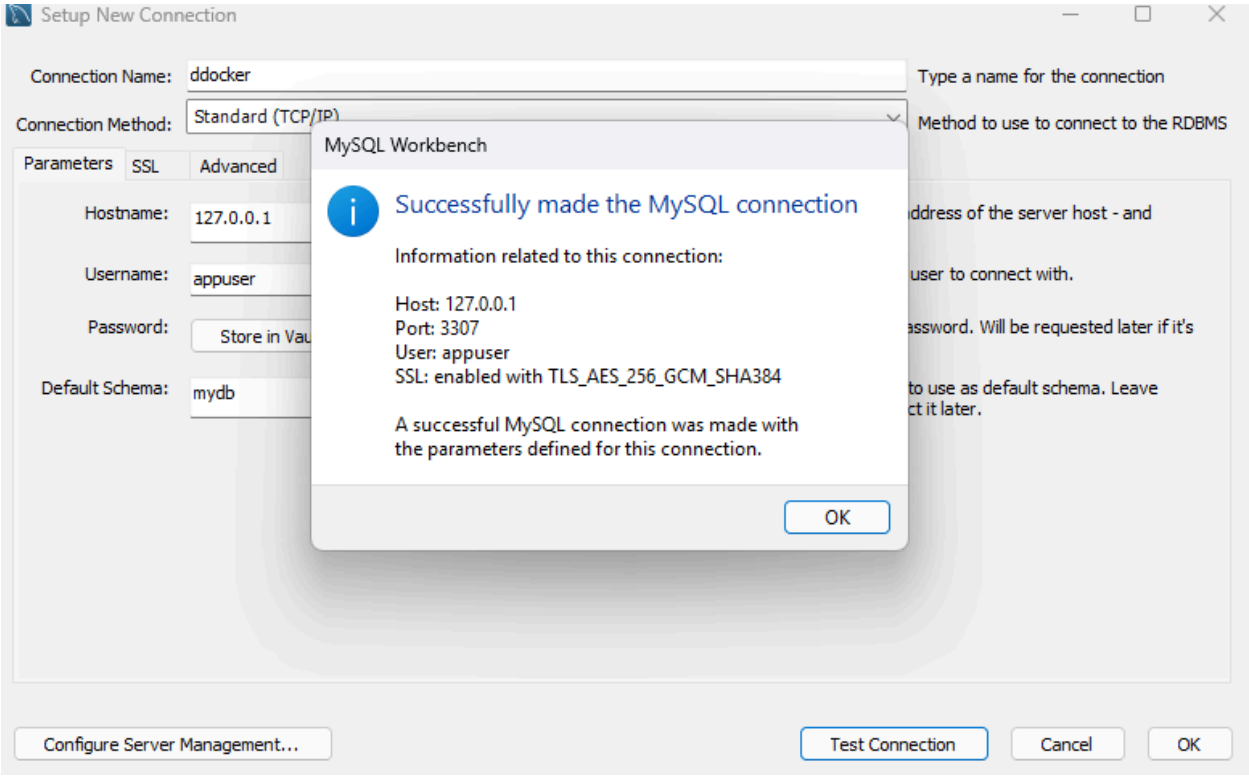
Configuration used:

Database: mydb

Username: appuser

Password: apppass

The MySQL service runs independently inside its own container.



3. Redis Service

- **Image used:** `redis:latest`
- **Container name:** `redis_cache`
- **Port:** `6379:6379`

Redis is an in-memory data store used for caching, session management, and fast data retrieval. It improves application performance by storing frequently accessed data in memory.

```
C:\Users\15L\docker-compose-demo>docker compose exec redis redis-cli
127.0.0.1:6379> ping
PONG
127.0.0.1:6379>
```

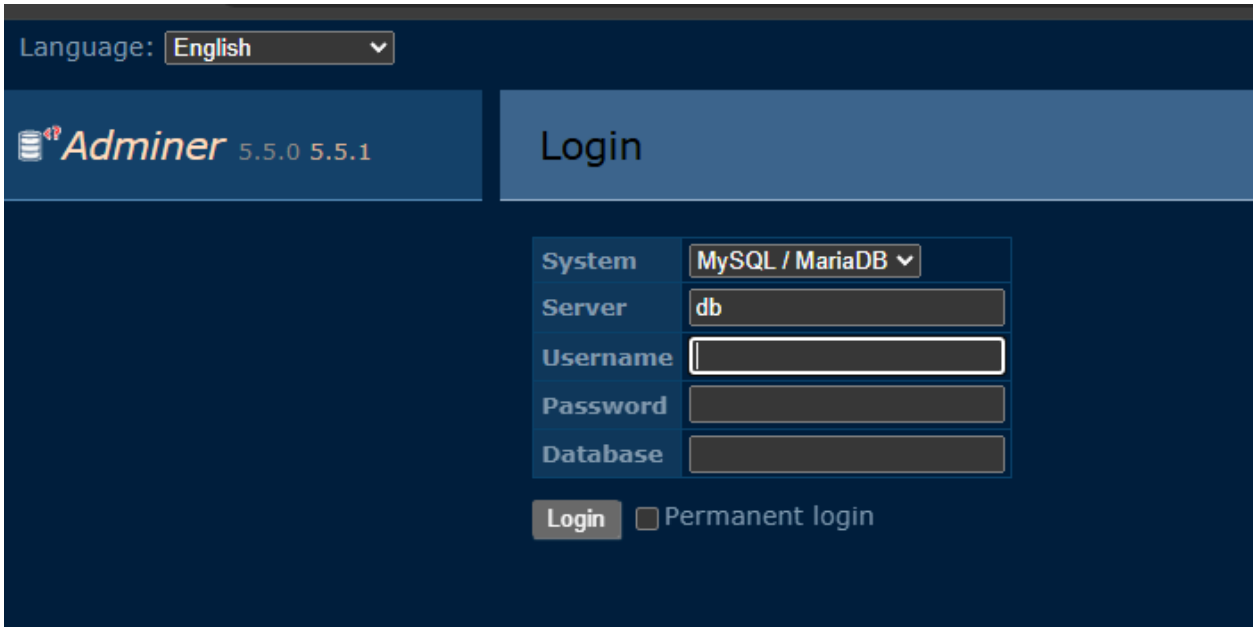
4. Adminer Service

- **Image used:** `adminer:latest`
- **Container name:** `adminer_ui`
- **Port:** `8085:8080`

Adminer provides a web-based interface to manage databases. It allows users to connect to and manage the MySQL database through a browser.

Access URL:

http://localhost:8085



What is Docker Compose?

Docker Compose is a tool that allows users to define, configure, and run multiple Docker containers together using a single YAML file.

Instead of manually creating and managing each container separately, Docker Compose allows multiple services to be started, stopped, and controlled using simple commands.

The configuration is written in a file called:

docker-compose.yml

This YAML file contains:

- Service names
- Docker images
- Container configurations
- Port mappings
- Environment variables
- Network settings
- Volume settings

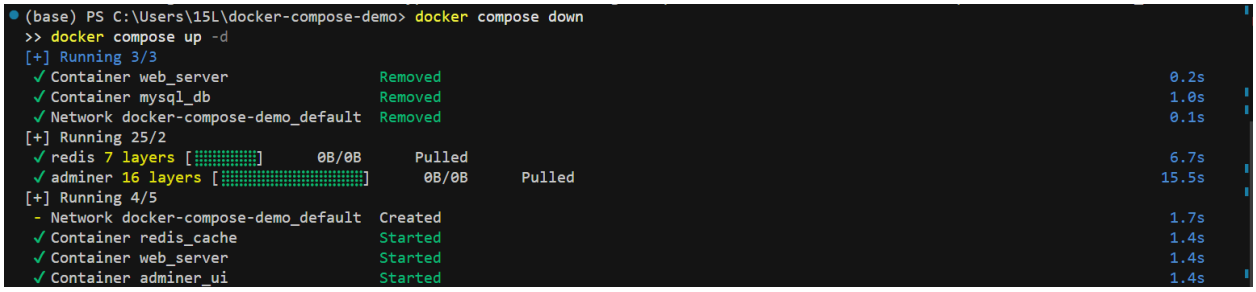
Using Docker Compose, multiple services such as web servers, databases, caches, and application servers can work together as a single application environment.

Commands

Docker compose up -d-Starts all services defined in the Docker Compose file and runs them in detached/background mode.

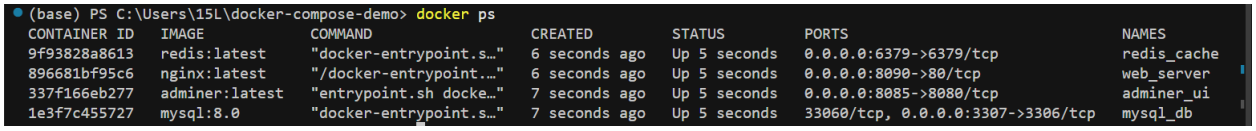
```
>> docker compose up -d
[+] Running 3/3
 ✓ Container web_server                  Removed
 ✓ Container mysql_db                    Removed
 ✓ Network docker-compose-demo_default   Removed
[+] Running 25/2
 ✓ redis 7 layers [⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿]      0B/0B      Pulled
 ✓ adminer 16 layers [⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿]      0B/0B      Pulled
[+] Running 4/5
 - Network docker-compose-demo_default   Created
 ✓ Container redis_cache                 Started
 ✓ Container web_server                  Started
 ✓ Container adminer_ui                  Started
```

Docker compose down-Stops and removes all containers and networks created by Docker Compose.



Docker ps-

Displays the list of currently running Docker containers.



Dockor compose ps-

Shows the status and details of containers managed by Docker Compose.

```
(base) PS C:\Users\15L\docker-compose-demo> docker compose ps
NAME          IMAGE            COMMAND                  SERVICE    CREATED         STATUS         PORTS
adminer_ui    adminer:latest   "entrypoint.sh docke…"   adminer    9 minutes ago   Up 9 minutes   0.0.0.0:8085->8080/tcp
mysql_db      mysql:8.0        "docker-entrypoint.s…"   database   9 minutes ago   Up 9 minutes   33060/tcp, 0.0.0.0:3307->3306/tcp
redis_cache   redis:latest     "docker-entrypoint.s…"   redis      9 minutes ago   Up 9 minutes   0.0.0.0:6379->6379/tcp
web_server    nginx:latest     "/docker-entrypoint.…"   web        9 minutes ago   Up 9 minutes   0.0.0.0:8090->80/tcp
```

Docker compose logs - Displays logs generated by all services in the Compose project.

```
(base) PS C:\Users\15L\docker-compose-demo> docker compose logs
adminer_ui  | [Mon Jul 27 09:35:26 2026] PHP 8.4.23 Development Server (http://[::]:8080) started
adminer_ui  | [Mon Jul 27 09:35:52 2026] [::ffff:172.23.0.1]:49504 Accepted
adminer_ui  | [Mon Jul 27 09:35:52 2026] [::ffff:172.23.0.1]:49504 [200]: GET /
adminer_ui  | [Mon Jul 27 09:35:52 2026] [::ffff:172.23.0.1]:49504 Closing
adminer_ui  | [Mon Jul 27 09:35:52 2026] [::ffff:172.23.0.1]:49512 Accepted
adminer_ui  | [Mon Jul 27 09:35:52 2026] [::ffff:172.23.0.1]:49512 [200]: GET /?file=default.css&version=5.5.0
adminer_ui  | [Mon Jul 27 09:35:52 2026] [::ffff:172.23.0.1]:49512 Closing
adminer_ui  | [Mon Jul 27 09:35:52 2026] [::ffff:172.23.0.1]:49524 Accepted
adminer_ui  | [Mon Jul 27 09:35:52 2026] [::ffff:172.23.0.1]:49524 [200]: GET /?file=dark.css&version=5.5.0
adminer_ui  | [Mon Jul 27 09:35:52 2026] [::ffff:172.23.0.1]:49524 Closing
adminer_ui  | [Mon Jul 27 09:35:52 2026] [::ffff:172.23.0.1]:49536 Accepted
adminer_ui  | [Mon Jul 27 09:35:52 2026] [::ffff:172.23.0.1]:49536 [200]: GET /?file=functions.js&version=5.5.0
adminer_ui  | [Mon Jul 27 09:35:52 2026] [::ffff:172.23.0.1]:49536 Closing
mysql_db    | 2026-07-27 09:35:26+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.46-1.el9 started.
mysql_db    | 2026-07-27 09:35:26+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
mysql_db    | 2026-07-27 09:35:26+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.46-1.el9 started.
mysql_db    | 2026-07-27 09:35:26+00:00 [Note] [Entrypoint]: Initializing database files
mysql_db    | 2026-07-27T09:35:26.372515Z 0 [Warning] [MY-011068] [Server] The syntax '--skip-host-cache' is deprecated and will be remo
ved in a future release. Please use SET GLOBAL host_cache_size=0 instead.
mysql_db    | 2026-07-27T09:35:26.372570Z 0 [System] [MY-013169] [Server] /usr/sbin/mysqld (mysqld 8.0.46) initializing of server in pro
gress as process 80
mysql_db    | 2026-07-27T09:35:26.376959Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
adminer_ui  | [Mon Jul 27 09:35:52 2026] [::ffff:172.23.0.1]:49552 Accepted
```

Docker compose logs -f-Continuously displays live logs of running services.

```
(base) PS C:\Users\15L\docker-compose-demo> docker compose logs -f
redis_cache  | Starting Redis Server
redis_cache  | 1:C 27 Jul 2026 09:35:26.182 * oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
redis_cache  | 1:C 27 Jul 2026 09:35:26.182 * Redis version=8.8.1, bits=64, commit=00000000, modified=1, pid=1, just started
redis_cache  | 1:C 27 Jul 2026 09:35:26.182 * Configuration loaded
redis_cache  | 1:M 27 Jul 2026 09:35:26.182 * monotonic clock: POSIX clock_gettime
redis_cache  | 1:M 27 Jul 2026 09:35:26.183 * Running mode=standalone, port=6379.
redis_cache  | 1:M 27 Jul 2026 09:35:26.183 * <bf> RedisBloom version 8.8.2 (Git=unknown)
redis_cache  | 1:M 27 Jul 2026 09:35:26.183 * <bf> Registering configuration options: [
redis_cache  | 1:M 27 Jul 2026 09:35:26.183 * <bf>      { bf-error-rate       :      0.01 }
redis_cache  | 1:M 27 Jul 2026 09:35:26.183 * <bf>      { bf-initial-size     :       100 }
redis_cache  | 1:M 27 Jul 2026 09:35:26.183 * <bf>      { bf-expansion-factor :         2 }
redis_cache  | 1:M 27 Jul 2026 09:35:26.183 * <bf>      { cf-bucket-size      :         2 }
redis_cache  | 1:M 27 Jul 2026 09:35:26.183 * <bf>      { cf-initial-size     :      1024 }
redis_cache  | 1:M 27 Jul 2026 09:35:26.183 * <bf>      { cf-max-iterations   :        20 }
redis_cache  | 1:M 27 Jul 2026 09:35:26.183 * <bf>      { cf-expansion-factor :         1 }
redis_cache  | 1:M 27 Jul 2026 09:35:26.183 * <bf>      { cf-max-expansions   :        32 }
redis_cache  | 1:M 27 Jul 2026 09:35:26.183 * <bf> ]
redis_cache  | 1:M 27 Jul 2026 09:35:26.183 * Module 'bf' loaded from /usr/local/lib/redis/modules//redisbloom.so
redis_cache  | 1:M 27 Jul 2026 09:35:26.186 * <search> search-workers default: 16 (min of MAX_WORKER_THREADS=16 and CPU cores)
redis_cache  | 1:M 27 Jul 2026 09:35:26.186 * <search> Redis version found by RedisSearch : 8.8.1 - oss
redis_cache  | 1:M 27 Jul 2026 09:35:26.186 * <search> RediSearch version 8.8.0 (Git=d2ae026)
redis_cache  | 1:M 27 Jul 2026 09:35:26.186 * <search> Low level api version 1 initialized successfully
redis_cache  | 1:M 27 Jul 2026 09:35:26.186 * <search> gc: ON, prefix min length: 2, min word length to stem: 4, prefix max expansions:
200, query timeout (ms): 500, timeout policy: return, oom policy: return, cursor read size: 1000, cursor max idle (ms): 300000, max doct
able size: 1000000, max number of search results:  1000000, default scorer: BM25STD,
redis_cache  | 1:M 27 Jul 2026 09:35:26.186 * <search> Initialized thread pools!
redis_cache  | 1:M 27 Jul 2026 09:35:26.186 * <search> Enabled workers threadpool of size 16
redis_cache  | 1:M 27 Jul 2026 09:35:26.191 * <search> Subscribe to config changes
redis_cache  | 1:M 27 Jul 2026 09:35:26.191 * <search> Subscribe to cluster slot migration events
redis_cache  | 1:M 27 Jul 2026 09:35:26.191 * <search> Enabled role change notification
redis_cache  | 1:M 27 Jul 2026 09:35:26.192 * <search> Cluster configuration: AUTO partitions, type: 0, coordinator timeout: 0ms
```

Docker compose stop-

Stops running Compose services without removing the containers.

```
(base) PS C:\Users\15L\docker-compose-demo> docker compose stop
[+] Stopping 4/4
 ✓ Container web_server   Stopped
 ✓ Container redis_cache  Stopped
 ✓ Container adminer_ui   Stopped
 ✓ Container mysql_db     Stopped
```

Docker compose start-

Starts previously stopped Docker Compose containers.

```
(base) PS C:\Users\15L\docker-compose-demo> docker compose start
[+] Running 4/4
 ✓ Container redis_cache  Started
 ✓ Container mysql_db     Started
 ✓ Container web_server   Started
 ✓ Container adminer_ui   Started
```

Docker compose restart-Restarts all containers managed by Docker Compose.

```
(base) PS C:\Users\15L\docker-compose-demo> docker compose restart
[+] Restarting 4/4
 ✓ Container adminer_ui   Started
 ✓ Container redis_cache  Started
 ✓ Container mysql_db     Started
 ✓ Container web_server   Started
```

Docker compose down -v-

Stops and removes containers, networks, and associated volumes.

```
(base) PS C:\Users\15L\docker-compose-demo> docker compose down -v
[+] Running 5/5
 ✓ Container web_server                  Removed
 ✓ Container adminer_ui                  Removed
 ✓ Container redis_cache                 Removed
 ✓ Container mysql_db                    Removed
 ✓ Network docker-compose-demo_default   Removed
```

Docker compose images-

Lists the Docker images used by the Compose services.

```
(base) PS C:\Users\15L\docker-compose-demo> docker compose images
CONTAINER           REPOSITORY          TAG                 IMAGE ID            SIZE
```

Docker compose config-Validates and displays the configuration of the Docker Compose YAML file.

```
(base) PS C:\Users\15L\docker-compose-demo> docker compose config
name: docker-compose-demo
services:
  adminer:
    container_name: adminer_ui
    depends_on:
      database:
        condition: service_started
        required: true
    image: adminer:latest
    networks:
      default: null
    ports:
      - mode: ingress
        target: 8080
        published: "8085"
        protocol: tcp
  database:
    container_name: mysql_db
    environment:
      MYSQL_DATABASE: mydb
      MYSQL_PASSWORD: apppass
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_USER: appuser
    image: mysql:8.0
    networks:
      default: null
    ports:
      - mode: ingress
        target: 3306
        published: "3307"
        protocol: tcp
  redis:
    container_name: redis_cache
```

Conclusion

In this experiment, Docker Compose was used to create and manage multiple services using a single `docker-compose.yml` configuration file. Different services such as **Nginx web server, MySQL database, Redis cache, and Adminer database management tool** were created and executed as separate Docker containers.

Docker Compose simplified the process of configuring, starting, stopping, and managing multiple containers together. The services were successfully connected through Docker's internal network, allowing them to work as a complete application environment.

Through this experiment, we learned how to create containers using Docker images, manage services using Docker Compose commands, handle port mappings, view logs, and control the

lifecycle of containers. Docker Compose provides an efficient way to develop, test, and deploy multi-container applications in a consistent and organized manner.